

System Overview

[illegible]

Data Model

Core Tables (Supabase / PostgreSQL)

[illegible]

[illegible]

API Design

Social Endpoints

[illegible]

```

POST /api/profile/verify-wallet
  → Verify wallet ownership (sign message)
GET /api/profile/:username/predictions
  → User's prediction history
GET /api/profile/:username/stats
  → Detailed stats (P&L chart, accuracy by category)

# Follow
POST /api/follow/:userId
  → Follow a user
DELETE /api/follow/:userId
  → Unfollow
GET /api/profile/:username/followers
  → Follower list (paginated)
GET /api/profile/:username/following
  → Following list (paginated)

# Predictions
POST /api/predictions
  → Create prediction (with optional trade)
GET /api/predictions/:id
  → Single prediction with comments
PUT /api/predictions/:id
  → Update thesis / close position
DELETE /api/predictions/:id
  → Delete (only if no engagement)
GET /api/predictions/market/:slug
  → All predictions on a market

# Feed
GET /api/feed
  → Personalized feed (auth required)
GET /api/feed/trending
  → Trending predictions (public)
GET /api/feed/latest
  → Chronological public feed
GET /api/feed/market/:slug
  → Market-specific feed

# Comments
POST /api/comments
  → Post comment (on prediction or market)
GET /api/comments/prediction/:id
  → Comments on a prediction
GET /api/comments/market/:slug
  → Market discussion thread
DELETE /api/comments/:id
  → Delete own comment

# Reactions
POST /api/reactions
  → React to prediction/comment
DELETE /api/reactions/:targetType/:targetId
  → Remove reaction

# Leaderboards
GET /api/leaderboard/:period/:category
  → Weekly/monthly/alltime rankings
GET /api/leaderboard/badges/:username
  → User's badges

```

Feed Generation Logic

```

When user posts a prediction:
  1. INSERT into predictions
  2. For each follower F of U:
    INSERT into feed_items (user_id=F, actor_id=U, action='prediction', ...)
  3. INSERT into feed_items for the market; feed (market_slug)
  4. If whale-sized ($10K+):
    INSERT into feed_items for ALL users following that market; category
  5. Fire notification to followers with notifications enabled

When market resolves:
  1. UPDATE all active predictions on that market
  2. Compute P&L for each prediction
  3. UPDATE author profile stats (pnl, win_rate, brier_score)
  4. Fire notifications to authors
  5. Recompute affected leaderboards

```

Real-time Architecture

```

Supabase Realtime (WebSocket)
  ■ Channel: feed: {userId}
  ■ New feed items pushed in real-time
  ■ Channel: market: {slug}
  ■ New predictions, comments, price changes
  ■ Channel: notifications: {userId}
  ■ New notification badge updates
  ■ Channel: leaderboard: {period}: {category}
  ■ Live rank changes during active periods

```

For the initial build, Supabase Realtime handles all WebSocket needs. If we outgrow it (>10K concurrent connections), migrate to a dedicated Socket.IO or Ably layer.

Wallet Verification Flow

To prevent fake P&L claims, wallet ownership must be cryptographically verified:

```

1. User connects wallet (MetaMask / WalletConnect)
2. Frontend requests a challenge: GET /api/profile/verify-challenge
   → Server returns: {message: "HYPERFLEX verify wallet",
   ; 0x...at1710000000", nonce: "abc123"}
3. User signs the message with their private key (MetaMask popup)
4. Frontend sends: POST /api/profile/verify-wallet {signature, nonce}
5. Server:
   a. Recovers signer address from signature using ethers.verifyMessage()
   b. Confirms it matches the claimed address
   c. Sets wallets_verified = true on profile
   d. Fetches historical positions from Polymarket CLOB for address
   e. Backfills prediction history and computes initial P&L stats
6. Profile now shows "Verified" badge with real on-chain P&L

```

Prediction Lifecycle

[illegible]

Build Phases

Phase 1: Social Foundation (Weeks 1-4)

Goal: Users can create profiles, follow each other, post predictions, and browse a feed.

Task	Priority	Effort	Dependencies
Profiles table + API (CRUD)	P0	3 days	Existing auth system
Username registration flow	P0	1 day	Profiles
Follow/unfollow API	P0	2 days	Profiles

Task	Priority	Effort	Dependencies
Predictions table + API	P0	4 days	Profiles
Basic feed (chronological)	P0	3 days	Predictions, Follows
Profile page UI	P0	3 days	Profile API
Prediction card component	P0	2 days	Predictions API
Feed page UI	P0	3 days	Feed API
Post prediction UI (tied to trade)	P0	3 days	Predictions API
Wallet verification	P1	3 days	ethers.js
Total		~4 weeks	

Deliverable: A working social feed where users post predictions linked to real markets. Every prediction is a piece of content in the feed.

Phase 2: Engagement & Discussion (Weeks 5-8)

Goal: Markets become discussion hubs. Predictions get reactions and comments. Notifications keep users coming back.

Task	Priority	Effort	Dependencies
Comments API + UI (threaded)	P0	4 days	Predictions
Reactions API + UI	P0	2 days	Predictions, Comments
Market discussion page	P0	3 days	Comments
Notification system (in-app)	P0	4 days	Follow, Comments, Reactions
Share prediction (Twitter card)	P1	2 days	Predictions
Trending feed algorithm	P1	3 days	Reactions, Feed
Search (users + markets)	P1	3 days	Full-text search
Total		~3 weeks	

Deliverable: Markets have discussion threads. Users get notified when someone reacts to or comments on their predictions. Share buttons generate Twitter cards for viral distribution.

Phase 3: Reputation & Leaderboards (Weeks 9-12)

Goal: Prediction accuracy is tracked and ranked. Top predictors gain status. Trust is verifiable.

Task	Priority	Effort	Dependencies
P&L computation engine	P0	4 days	Predictions, CLOB data
Win rate / Brier score calculator	P0	3 days	Resolved predictions
Leaderboard snapshot cron	P0	2 days	Stats engine
Leaderboard UI (weekly/monthly/alltime)	P0	3 days	Leaderboard API
Badge system (whale, top1%, etc.)	P1	2 days	Stats
Historical P&L backfill for verified wallets	P1	3 days	Wallet verification
Profile stats page (charts, breakdown)	P1	3 days	Stats engine
Total		~3 weeks	

Deliverable: Leaderboards show the best predictors by category. Profiles display verified track records. Badges signal expertise and volume.

Phase 4: Monetization & Groups (Weeks 13-18)

Goal: Revenue flows. Creators can monetize. Premium features justify subscriptions.

Task	Priority	Effort	Dependencies
Stripe integration (Pro/Alpha tiers)	P0	4 days	—
Subscription gating middleware	P0	2 days	Stripe
Pro features: unlimited predictions, alerts	P0	3 days	Subscriptions

Task	Priority	Effort	Dependencies
Alpha features: API access, copy-trading	P1	5 days	Subscriptions
Groups (create, join, post)	P1	5 days	Social foundation
Paid groups (Stripe Connect for creators)	P2	4 days	Groups, Stripe
Telegram bot (free/paid tiers)	P1	5 days	Signal engine
Email digest (weekly top predictions)	P2	2 days	Feed
Total		~5 weeks	

Deliverable: Users can subscribe to Pro/Alpha. Creators can create paid groups. Revenue is flowing.

Tech Stack

Layer	Technology	Why
Backend	Node.js / Express (existing)	Already built, extend with new routes
Database	Supabase (PostgreSQL)	Already integrated, RLS, Realtime built-in
Cache	Supabase or Upstash Redis	Feed cache, leaderboards, rate limiting
Auth	Existing JWT + wallet signing	Extend with wallet verification
Hosting	Railway (existing)	Auto-deploy on push
CDN	Cloudflare	Already using for trade proxy
Payments	Stripe + Stripe Connect	Subscriptions + creator payouts
Real-time	Supabase Realtime	WebSocket channels for feed/notifications
Image storage	Supabase Storage or Cloudflare R2	Avatars, banners

Layer	Technology	Why
Email	Resend or Postmark	Notifications, digests, onboarding
Telegram	node-telegram-bot-api	Signal bot delivery

Security Considerations

Concern	Mitigation
Fake P&L	Wallet verification + on-chain position proof
Spam predictions	Rate limiting (free: 3/day, Pro: unlimited)
Harassment	Report system, content moderation, block users
Data scraping	Rate limiting on public APIs, API keys for bulk access
Wallet security	Never store private keys; signature-only verification
Content manipulation	Predictions immutable once posted (can close, not edit side/price)

Migration Path from Current Architecture

HYPERFLEX today is a monolithic Express app with HTML pages. The social layer builds on top without requiring a rewrite:

```
Current:
server.js (Express) → public/*.html (vanilla JS)
Supabase (users, creators, rewards)

After social layer:
server.js (Express)
  Existing routes (unchanged)
  /api/polymarket/* (trading)
  /api/alpha/* (signals)
  /api/admin/* (admin panel)
  /api/screener/* (market data)
  New social routes (added)
  /api/profile/*
  /api/predictions/*
  /api/feed/*
  /api/comments/*
```


We don't compute rankings on every request. A cron job runs hourly (or daily) and snapshots the top N users by category into `leaderboard_snapshots`. The API serves the latest snapshot. Fast reads, acceptable staleness.

5. Start with server-rendered HTML, not a SPA framework

Consistent with the existing architecture. No React/Vue overhead. Vanilla JS + HTML templates. Fast to build, fast to ship, easy to iterate. Move to a framework later if/when complexity demands it.